

```

1  import paramiko
2  import os
3  import time
4  import sys
5
6  # Importações da biblioteca Rich
7  from rich.console import Console
8  from rich.panel import Panel
9  from rich.table import Table
10 from rich.layout import Layout
11 from rich.text import Text
12 from rich.live import Live
13 from rich.columns import Columns
14
15 # =====
16 # CONFIGURAÇÕES DE ACESSO AO SERVIDOR LINUX (ASH/BUSYBOX)
17 # =====
18 SSH_HOST = "172.16.35.2"
19 SSH_USER = "root"
20 SSH_PASS = "h4rdFlip"
21
22 # Configuração de escala para o gráfico de rede (Ex: 10 MB/s equivale a 100% da barra)
23 REDE_MAX_VELOCIDADE_MBS = 100.0
24
25 console = Console()
26 ULTIMO_ERRO = ""
27
28 # Variáveis globais para calcular a velocidade da rede
29 g_bytes_in_ant = 0.0
30 g_bytes_out_ant = 0.0
31 g_tempo_ant = 0.0
32
33 def obter_metricas_servidor(ssh):
34     """Executa e extrai todas as métricas cruas de forma isolada e segura"""
35     global ULTIMO_ERRO
36     try:
37         # 1. Captura estritamente a linha GLOBAL da cpu (ignora cpu0, cpu1, etc.)
38         stdin, stdout, stderr = ssh.exec_command("cat /proc/stat | grep '^cpu '")
39         cpu1_linha = stdout.readline().strip().split()
40
41         time.sleep(0.5) # Janela de amostragem para variação de ciclos
42
43         # 2. Captura o segundo estado da CPU de forma isolada
44         stdin, stdout, stderr = ssh.exec_command("cat /proc/stat | grep '^cpu '")
45         cpu2_linha = stdout.readline().strip().split()
46
47         # 3. Executa as demais métricas sem misturar com o arquivo /proc/stat
48         comando_demais = (
49             "cat /proc/sys/kernel/hostname ; "
50             "cat /proc/meminfo | grep -E 'MemTotal|MemAvailable|MemFree' | awk '{print "
51             "$2}' | tr '\\n' '| ' ; echo ; "
52             "df -h / | awk 'NR==2 {print $5}' || df -h / | awk 'NR==3 {print $4}' ; "
53             "interface=$(ip route | grep default | awk '{print $5}'); "
54             "cat /proc/net/dev | grep \"\$interface\" | awk '{print $2\\\"|\\\"$10}'"
55         )
56
57         stdin, stdout, stderr = ssh.exec_command(comando_demais)
58         linhas_bloco = [l.strip() for item in stdout.readlines() if (l := item.strip())]
59
60         if len(linhas_bloco) < 4:
61             ULTIMO_ERRO = "Resposta incompleta do servidor nas métricas de hardware."
62             return None
63
64         # --- Processamento de CPU ---
65         uso_cpu = 0.0
66         if cpu1_linha and cpu2_linha and cpu1_linha[0] == 'cpu' and cpu2_linha[0] ==
        'cpu':

```

```

66         c1_v = [int(x) for x in cpu1_linha[1:]]
67         c2_v = [int(x) for x in cpu2_linha[1:]]
68         diff_total = sum(c2_v) - sum(c1_v)
69         diff_idle = c2_v[3] - c1_v[3] # Índice 3 é o tempo ocioso (idle)
70         if diff_total > 0:
71             uso_cpu = round((1.0 - (diff_idle / diff_total)) * 100, 2)
72
73     # --- Processamento Seguro das Demais Linhas (Garantindo os Índices Fixos) ---
74     hostname = linhas_bloco[0]
75
76     # Memória (Linha 1)
77     dados_mem = linhas_bloco[1].split('|')
78     mem_total = float(dados_mem[0])
79     # Se o sistema compactado não tiver MemAvailable (índice 1 vazio), usa MemFree
80     # (índice 2)
81     mem_livre = float(dados_mem[1]) if len(dados_mem) > 2 and dados_mem[1] else float(
82     (dados_mem[2])
83     perc_mem = round(((mem_total - mem_livre) / mem_total) * 100, 2)
84     tam_memoria = f"{mem_total / (1024**2):.2f} GB"
85
86     # Disco (Linha 2)
87     perc_disco = float(linhas_bloco[2].replace('%', ''))
88
89     # Rede (Linha 3)
90     bytes_in_raw, bytes_out_raw = linhas_bloco[3].split('|')
91
92     return {
93         "hostname": hostname,
94         "uso_cpu": uso_cpu,
95         "uso_memoria": perc_mem,
96         "tam_memoria": tam_memoria,
97         "perc_disco": perc_disco,
98         "bytes_in": float(bytes_in_raw),
99         "bytes_out": float(bytes_out_raw)
100     }
101 except Exception as e:
102     ULTIMO_ERRO = f"Erro na coleta: {e}"
103     return None
104
105 def coletar_dados_com_velocidade():
106     global g_bytes_in_ant, g_bytes_out_ant, g_tempo_ant, ULTIMO_ERRO
107
108     ssh = paramiko.SSHClient()
109     ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
110
111     try:
112         ssh.connect(SSH_HOST, username=SSH_USER, password=SSH_PASS, timeout=3)
113         res = obter_metricas_servidor(ssh)
114
115         if not res:
116             return None
117
118         tempo_atual = time.time()
119         vel_in_mbs = 0.0
120         vel_out_mbs = 0.0
121
122         if g_tempo_ant > 0:
123             diff_tempo = tempo_atual - g_tempo_ant
124             if diff_tempo > 0:
125                 vel_in_mbs = ((res["bytes_in"] - g_bytes_in_ant) / (1024**2)) /
126                 diff_tempo
127                 vel_out_mbs = ((res["bytes_out"] - g_bytes_out_ant) / (1024**2)) /
128                 diff_tempo
129
130         vel_in_mbs = max(0.0, vel_in_mbs)
131         vel_out_mbs = max(0.0, vel_out_mbs)

```

```

129     g_bytes_in_ant = res["bytes_in"]
130     g_bytes_out_ant = res["bytes_out"]
131     g_tempo_ant = tempo_atual
132
133     perc_grafico_in = min(100.0, (vel_in_mbs / REDE_MAX_VELOCIDADE_MBS) * 100)
134     perc_grafico_out = min(100.0, (vel_out_mbs / REDE_MAX_VELOCIDADE_MBS) * 100)
135
136     return {
137         "IP": SSH_HOST,
138         "HOSTNAME": res["hostname"],
139         "USO_CPU": res["uso_cpu"],
140         "USO_MEMORIA": res["uso_memoria"],
141         "TAMANHO_MEMORIA": res["tam_memoria"],
142         "DISCO_ROOT_PERC": res["perc_disco"],
143         "REDE_IN_MBS": round(vel_in_mbs, 2),
144         "REDE_OUT_MBS": round(vel_out_mbs, 2),
145         "PERC_GRAFICO_IN": round(perc_grafico_in, 1),
146         "PERC_GRAFICO_OUT": round(perc_grafico_out, 1),
147         "REDE_TOTAL_IN_MB": round(res["bytes_in"] / (1024**2), 1),
148         "REDE_TOTAL_OUT_MB": round(res["bytes_out"] / (1024**2), 1)
149     }
150 except Exception as e:
151     ULTIMO_ERRO = f"Falha de conexão: {e}"
152     return None
153 finally:
154     ssh.close()
155
156 def desenhar_barra(porcentagem, texto_sufixo, cor):
157     largura_maxima = 18
158     blocos_preenchidos = int((porcentagem / 100) * largura_maxima)
159     blocos_vazios = largura_maxima - blocos_preenchidos
160
161     barra = f"[{cor}]" + "█" * blocos_preenchidos + "[/]"
162     barra += "[grey37]" + "░" * blocos_vazios + "[/]"
163     return f"{barra} {texto_sufixo}"
164
165 def gerar_layout_dashboard(dados):
166     header_text = Text("📊 DASHBOARD INTEGRADO DE TELEMETRIA (REDE E HARDWARE)", style=
167 "bold white on navy_blue", justify="center")
168     header_panel = Panel(header_text, style="blue", padding=(0, 1))
169
170     if not dados:
171         erro_formatado = f"\n⚠ Sincronizando dados com o servidor remoto...\n\n[bold
172 yellow>Status:[/bold yellow] {ULTIMO_ERRO}\n"
173         return Panel(Text(erro_formatado, style="bold red", justify="center"), title=
174 "Status do Sistema", border_style="red")
175
176     cor_cpu = "green" if dados["USO_CPU"] < 50 else "yellow" if dados["USO_CPU"] < 80
177 else "red"
178     cor_mem = "green" if dados["USO_MEMORIA"] < 70 else "yellow" if dados["USO_MEMORIA"]
179 < 90 else "red"
180     cor_disco = "green" if dados["DISCO_ROOT_PERC"] < 75 else "yellow" if dados[
181 "DISCO_ROOT_PERC"] < 90 else "red"
182
183     # --- COLUNA DA ESQUERDA: HARDWARE ---
184     tab_hardware = Table(show_header=False, expand=True, box=None)
185     tab_hardware.add_row("[bold white]Uso de CPU[/bold white]")
186     tab_hardware.add_row(desenhar_barra(dados["USO_CPU"], f"[bold {cor_cpu}]{dados[
187 'USO_CPU']}%[/]", cor_cpu))
188     tab_hardware.add_row("")
189     tab_hardware.add_row("[bold white]Uso de Memória RAM[/bold white]")
190     tab_hardware.add_row(desenhar_barra(dados["USO_MEMORIA"], f"[bold {cor_mem}]{dados[
191 'USO_MEMORIA']}%[/]", cor_mem))
192     tab_hardware.add_row("")
193     tab_hardware.add_row("[bold white]Armazenamento (Disco /)[/bold white]")
194     tab_hardware.add_row(desenhar_barra(dados["DISCO_ROOT_PERC"], f"[bold {cor_disco}]{
195 dados['DISCO_ROOT_PERC']}%[/]", cor_disco))

```

```

187
188 painel_hardware = Panel(tab_hardware, title="🌐 Recursos do Sistema", border_style=
"cyan")
189
190 # --- COLUNA DA DIREITA: REDE E DETALHES ---
191 tab_rede = Table(show_header=False, expand=True, box=None)
192 tab_rede.add_row(f"[bold cyan]Host:[/bold cyan] [bold magenta]{dados['HOSTNAME']}
[/bold magenta] | [bold cyan]IP:[/bold cyan] {dados['IP']}")
193 tab_rede.add_row(f"[bold cyan]Memória Total Disponível:[/bold cyan] {dados[
'TAMANHO_MEMORIA']}")
194 tab_rede.add_row("")
195
196 tab_rede.add_row("[bold green]📶 Velocidade de Download[/bold green]")
197 tab_rede.add_row(desenhar_barra(dados["PERC_GRAFICO_IN"], f"[bold green]{dados[
'REDE_IN_MBS']} MB/s[/]", "green"))
198 tab_rede.add_row(f"[dim white]Total acumulado recebido: {dados['REDE_TOTAL_IN_MB']}
MB[/dim white]")
199 tab_rede.add_row("")
200
201 tab_rede.add_row("[bold orange3]📶 Velocidade de Upload[/bold orange3]")
202 tab_rede.add_row(desenhar_barra(dados["PERC_GRAFICO_OUT"], f"[bold orange3]{dados[
'REDE_OUT_MBS']} MB/s[/]", "orange3"))
203 tab_rede.add_row(f"[dim white]Total acumulado enviado: {dados['REDE_TOTAL_OUT_MB']}
MB[/dim white]")
204
205 painel_rede = Panel(tab_rede, title="🌐 Tráfego de Rede Real", border_style="magenta"
)
206
207 corpo_dashboard = Columns([painel_hardware, painel_rede], expand=True)
208
209 layout_final = Layout()
210 layout_final.split_column(
211     Layout(header_panel, size=3),
212     Layout(corpo_dashboard)
213 )
214 return layout_final
215
216 # =====
217 # LAÇO PRINCIPAL DE ATUALIZAÇÃO
218 # =====
219 if __name__ == "__main__":
220     os.system('cls' if os.name == 'nt' else 'clear')
221
222     # Mensagem informativa inicial na parte inferior
223     console.print("[bold yellow]Pressione Ctrl+C para encerrar o Painel a qualquer
momento.[/bold yellow]\n")
224     time.sleep(1)
225
226     try:
227         with Live(gerar_layout_dashboard(None), refresh_per_second=2, screen=True) as
live:
228             while True:
229                 dados_atuais = coletar_dados_com_velocidade()
230                 live.update(gerar_layout_dashboard(dados_atuais))
231                 time.sleep(0.3)
232     except KeyboardInterrupt:
233         # Captura o Ctrl+C de forma limpa no terminal Linux
234         pass
235
236     print("\n[bold green]Painel encerrado com sucesso![/bold green]")
237

```